# CSC3002 | Tutorial review and revision priorities

## Tutorial: 26 October 2026 | Midterm: Wednesday, 28 October

The strongest preparation is to practise three tasks: identify whether code compiles, trace exactly what changes, and explain whether the result remains valid. Your TA's photographed examples repeatedly use short snippets with one important type or state distinction.

### What is confirmed, and what is a study recommendation?

- **Your reported exam format:** Part I Q1-Q4, single choice, 20 marks; Part II Q5-Q11, multiple choice, 40 marks; Part III Q12-Q14, written response, 40 marks.

- **Your reported exclusion:** no questions requiring written hexadecimal memory addresses. Continue studying pointers through element indices, values, types, bounds and lifetime.

- **Photographed tutorial coverage:** 25 photos containing 23 distinct slides, numbered 3-25 of 26. Slides 18 and 19 appear twice. Slides 1, 2 and 26 are unavailable; their contents are not inferred.

- **Your contemporaneous notes:** especially emphasize function pointers, why virtual/abstract classes exist, writing operators and explaining const, and writing/tracing templates.

- **Broader revision scope:** Lectures 1-9, as recorded in your existing revision guide. Topics absent from the photographed tutorial are not automatically excluded.

- **Mock design choices:** individual question weights, 100-minute practice timing and marking policy are practice conventions, not official exam rules. The three section totals and question counts follow your message exactly.

### What to prioritise now

1. **Parameters and pointers:** value vs reference vs copied pointer; caller state; pointer movement vs pointee mutation; valid bounds.

2. **Classes and operators:** const members, static shared state, virtual dispatch, abstract interfaces, writing a small operator overload.

3. **Templates and arithmetic:** deduced type, expression type, declared return type, then caller type. Integer division is especially important for you.

4. **Streams and memory lifetime:** established gaps in your previous practice; retain them even though they are not on the available review slides.

5. **Strings, overloads/defaults and short-circuiting:** quick marks if you follow the precise rules.

### Evidence behind your personal priorities

Your earlier graded mock recorded incorrect answers on array parameters, one-past-end pointers, integer division, reference mutation, delete[], const pointers, array reassignment and pointer increments. Its 28/100 included 38 marks of unanswered work; blanks are not proof of a misconception. Later quiz follow-ups directly asked about array parameter sizes, a+4, ofstream, oss.str(), and fixed formatting. These are observed questions/gaps, not a recovered log of every interactive answer.

# 1 | Functions and evaluation

## Tutorial slides 3-11

### Use this four-step trace on every question

1. Name the types: array, pointer, reference, scalar, object or function pointer.

2. Parse the expression using parentheses and precedence; check short-circuiting.

3. Update caller variables and local copies separately.

4. State the result's type/value and whether the referenced object is still alive.

Keep **compile error**, **well-defined execution**, and **undefined behavior** separate. Compiling does not prove that execution is safe.

### Value, reference and pointer parameters

```
void update(int a, int& b, int* c) {
    a += 1; b += 2; *c += 3;
}
int x=4, y=5, z=6;
update(x, y, &z);       // x=4, y=7, z=9
```

`a` is a local copy; `b` aliases y; `c` is a local pointer copy that reaches z. Reassigning c would not reassign a caller pointer, but changing *c changes the pointed-to object. To change a caller's pointer itself, use a reference to pointer, such as int*&.

In the tutorial's grow example, n starts at 2 and becomes 6; m aliases b and becomes 5+6=11. The caller's a remains 2. Do not reuse a's unchanged value when calculating m inside the function.

### Overloading and defaults

```
void display(int value);
void display(double value, int precision=2);
```

`display(7)` chooses the int overload. `display(7.25)` chooses the double overload and uses precision=2. `display(7.25,1)` supplies both arguments. Defaults fill omitted trailing arguments; they do not automatically make an overload the best match. Return type alone cannot distinguish overloads. For example, f(int) and f(int,int=0) would make f(3) ambiguous.

### Function pointers: practise writing this

```
bool isEven(long n) { return n % 2 == 0; }
bool (*test)(long) = isEven;
bool ok = test(12L);    // true; cout prints 1 by default
```

Read from test outward: pointer to a function taking long and returning bool. `bool* test(long)` instead declares a function returning bool*. A callback parameter uses the same pattern: `bool (*pred)(long)`.

### Short-circuiting: fix this line in your notes

For flag=false, `flag && (++attempts > 0)` skips the increment because the left operand is false. `!flag || (++attempts > 0)` also skips it, because **!flag is true**, not false. attempts stays 0; the results are false and true. In the visits example, visits ends at 2, r1=true and r2=false. Skipped expressions perform no side effects.

Source: IMG_8041-IMG_8049; root C++ Concept Review.md; L1 expressions and L2/L6 functions.

# 2 | Arrays, pointers and strings

## Tutorial slides 12-16

### Think in indices, without numerical addresses

For `int values[5]={3,6,9,12,15}; int* pos=values+1;`, pos refers to index 1. `*(pos+2)` reads index 3 and gives 12. Computing pos+2 does not change pos. By contrast, ++pos changes the stored pointer.

The weights example starts at index 1, increments to index 2, then reads *(it+1) at index 3: result=8. it itself remains at index 2.

### Your highest-value pointer distinctions

| Expression | Effect |
|---|---|
| p=q | Changes which object p refers to |
| *p=*q | Copies the pointed-to value |
| *p++ | Reads old pointee; advances p |
| *++p | Advances p; reads new pointee |
| ++*p | Increments pointee; yields new value |
| (*p)++ | Yields old pointee value; increments pointee |

For each statement, record **pointer index, expression result, array contents**. Do not track only one of these.

### Array parameters and bounds: your quiz follow-ups

`void f(int a[6])` has a pointer parameter, not a six-element array parameter. The 6 does not enforce the caller's array length. An int[10] can be passed; the function still needs a length or other precondition to access safely. Inside f, sizeof(a) is sizeof(int*). In the caller, sizeof(actualArray) is the whole array's byte size. Element count is sizeof(array)/sizeof(array[0]) only while array is actually an array.

For int a[4], a+4 is a valid one-past endpoint, and (a+4)-a is 4 elements. Reading *(a+4) is undefined behavior. Even forming a+5 goes beyond the permitted pointer-arithmetic range. A raw array cannot be reassigned or incremented. int m[2][3] decays to a pointer to a three-int row, not int**.

### Strings: character count is not buffer size

```
char letter='G';
char label[]="Go";
std::string word="Blue";
// sizeof(letter)=1; sizeof(label)=3; strlen(label)=2
// word.substr(1,2) == "lu"; word remains "Blue"
```

C-string storage includes '\0'; strlen excludes it. `char b[8]` can hold at most seven content characters as a C string. strcpy copies the terminator too; it does not enlarge the destination. Two string literals cannot be joined with +; `std::string("C")+"++"` and `"C"+std::string("++")` both work. strcmp returns a negative/zero/positive result, not necessarily -1/0/1. std::string comparison compares contents; pointer equality does not perform a text comparison. find failure is std::string::npos.

Source: IMG_8050-IMG_8055; Tutorial 4 pp.3-11; L4/L6; prior quiz and grading feedback.

# 3 | Classes, const, static and polymorphism

## Tutorial slides 17-22

### Separate const from static

```
struct Ticket {
    const int code;
    static int created;
    Ticket(int c) : code(c) { ++created; }
    int getCode() const { return code; }
};
int Ticket::created=0;
```

After Ticket first(17), second(42), created is 2 and second.getCode() is 42. Each object has its own code; all objects share created. A const data member needs initialization, not later assignment. A const member function can be called on both const and non-const objects. It restricts modification through this; it does not require the caller to be const.

A static member function has no this pointer. It can use instance members only through an object. A local static retains its value across calls. These are different uses of static.

### Why virtual functions and abstract classes exist

A virtual function lets client code use one base interface while the concrete object's override supplies the behavior. In the tutorial, Vehicle& ref bound to a Scooter calls Scooter::kind(), giving "scooter". The reference's static type is Vehicle; the object's dynamic type is Scooter. A nonvirtual call is selected using static type. Access control and dispatch answer different questions.

A pure virtual declaration such as `virtual void report() const=0;` makes a class abstract if it has no non-pure final override for that operation. You cannot construct a Reporter object directly, but you can bind Reporter& to a concrete derived object. SilentReporter implements report() and can be instantiated. This enforces a common contract while leaving behavior to concrete types. An inherited concrete override can also satisfy the requirement; every descendant need not redeclare it.

Match the signature, including trailing const. `override` catches mismatches. For ordinary deletion of a derived object through a base pointer, give the base a virtual destructor.

### Understand every part of an operator signature

```
struct Money {
    int cents;
    Money operator+(const Money& other) const {
        return Money{cents + other.cents};
    }
};
```

- Return type Money: a new value, not a reference to a local object.

- const Money& other: borrow the right operand without copying it; do not modify it through other.

- Trailing const: do not modify ordinary left-operand state through this; permit a const left operand.

- ticket+fee calls ticket.operator+(fee). For 125 and 75, the result is 200 and the inputs stay unchanged.

A conventional += mutates the left operand and returns Money& via return *this. It should not have trailing const when modifying cents. However, the lecture's Point operator+ mutates the left operand: **trace the supplied body instead of assuming every + is non-mutating**. Know public/private defaults (struct/class), constructor initializer lists, copy construction vs assignment, and deep vs shallow copy.

Source: IMG_8056, 8059, 8061, 8063-8068; root notes; L7/L8; Tutorial 6.

# 4 | Templates and the full-scope safety net

## Tutorial slides 23-25

### Deduce, evaluate, convert, then assign

```
template<typename T>
T half(T value) { return value / 2; }
// half(7): T=int, int division, result int 3
// half(7.0): T=double, result double 3.5
// double x=half(7): x is 3.0; division already truncated
```

The tutorial also shows addOne(4)=5 and addOne(1.5)=2.5. Write the template declaration yourself, not just the calls. auto at the caller deduces the returned type; it does not repair truncation inside a function.

For `template<typename T> T Max(T a,T b)`, Max(1,1.1) fails deduction because the same T is inferred differently. Max<double>(1,1.1) works. With two type parameters but return type equal to the first, a selected double result can still convert to int on return. Required operations must exist for the chosen type; a generic comparison is not automatically valid for every class.

### Broader Lectures 1-9 checklist

The review photos are a priority guide, not proof that the rest is excluded. Use the existing revision guide for the following second pass.

| Lecture | Be able to do |
|---|---|
| L1 | Trace integer division, casts, scope, loops and short-circuiting; distinguish compilation/linking; know assertions |
| L2 | Trace value/reference calls; select overloads and defaults; write small functions such as swap/GCD |
| L3 | Separate interface/header from implementation; explain guards, enum values, seed reproducibility and local static initialization |
| L4 | Use C-string terminators/capacity, cctype and string search/substrings/modification; trace simple text loops |
| L5 | Trace formatting and unread input; use file/string streams and read-controlled loops |
| L6 | Read declarators/function pointers; trace arrays, aliases, increments and bounds; distinguish array vs pointer sizeof |
| L7 | Match allocation/deallocation; reason about lifetime, matrices, const/static and smart-pointer ownership |
| L8 | Write constructors/operators; understand copies, access control, virtual/abstract classes; revise Rational and CharStack/TokenScanner examples |
| L9 | Write function/class templates; explain deduction/return conversion; trace iterator and range-for loops and template selection sort |

### Iterators: a short but necessary follow-up

end() is one past the last element; do not dereference it. An empty range has begin()==end(). `for(auto x:v)` modifies a copy; `for(auto& x:v)` can modify the elements. Reverse iterators run from rbegin() to rend(); ++ moves toward earlier elements. Template member definitions generally need to be visible where instantiated. Standard stack/queue adaptors do not expose normal begin/end traversal; do not blindly transfer a custom course Stack example to std::stack.

Deprioritise literal-address calculation in old Q32 and the old mock's address table. Later-course hashing, trees, heaps, minimax and queue implementation were not developed in the reviewed L1-9 scope. C allocation API memorisation is explicitly downplayed on L7 p.20. These priorities are narrower than declaring every related concept excluded.

Source: IMG_8069-IMG_8071; L9; existing revision guide and coverage comparison.

# 5 | Your personal repair sheet

## Keep this beside your scratch paper

### Streams: your follow-up questions, answered precisely

- **ofstream:** writes to a file. cout writes to standard output. ostringstream accumulates text in memory; istringstream reads text from a string.

- **oss.str():** returns the accumulated string. Calling it alone neither prints nor saves it and does not empty oss. Assign it, send it to cout, or insert it into an open file stream.

- **fixed:** a formatting manipulator, not a container receiving the number. `oss << fixed << setprecision(2) << 3.14159;` stores text "3.14" in oss; the original numeric value does not change.

- **setw:** normally applies to the next formatted field only. fixed, precision, fill and alignment persist. Width is a minimum; it does not truncate.

- **Extraction vs getline:** >> normally skips leading whitespace and leaves the delimiter after the extracted value. getline consumes its delimiter without storing it. A pending newline can produce an empty string.

- **Failure:** clear() resets stream flags, not unread characters. If x caused numeric parsing to fail, clear() followed by the same numeric extraction fails again unless x is handled.

- **Loops:** prefer while(in >> x), while(getline(in,line)) or while(in.get(ch)). Testing !eof() before a read does not prove that read succeeds.

```
std::istringstream in("24\nAda Lovelace\n");
int id; std::string a,b;
in >> id;
std::getline(in,a);   // a == ""
std::getline(in,b);   // b == "Ada Lovelace"
```

### Allocation and copying: keep owner and observer separate

new int matches delete; new int[n] matches delete[], even when n is 1. Deleting an allocation does not erase the pointer variable or null every alias. After `int* q=p; delete p; p=nullptr;`, q still dangles. Do not read through q or delete it again. Returning a pointer/reference to a local automatic object also leaves a dangling result.

For a row-by-row matrix, delete[] each row before deleting the outer pointer array while it is still needed to find those rows. Deleting only the outer array leaks the rows. A contiguous flat matrix instead has one allocation.

A class with an owning raw pointer generally needs coherent destructor, copy constructor and copy assignment behavior (Rule of Three), or copying must be disallowed. A shallow copy shares the allocation; two destructors can double-delete it. `Buffer b=a;` constructs b; `b=a;` assigns an already existing b. Assignment must handle old destination storage and self-assignment.

unique_ptr has one owner and can move, not copy. shared_ptr copies share ownership; the last strong owner releases the object. weak_ptr observes without extending lifetime; a successful lock() creates a temporary strong owner. Borrowed raw pointers can still dangle.

### A practical written-response habit

Write the type and one intermediate state even if you cannot finish. On the prior mock, unanswered work was a large part of the lost marks. This mock provides partial-credit rubrics for written work so you can diagnose incomplete reasoning separately from an incorrect rule.

Source: earlier quiz follow-ups and graded mock; L5 pp.11-38; L7 pp.5-30; L8 copying.

# 6 | How to use the remaining preparation time

## A focused sequence before Wednesday

1. **35 minutes:** redo the tutorial's value/reference, short-circuit, pointer-offset and template examples with changed inputs. State both type and value.

2. **40 minutes:** write a callback declaration, Money +/+=, an abstract base and concrete override, and a function template from a blank page. Explain both const qualifiers aloud.

3. **35 minutes:** practise your pointer increment table, array parameter size, one-past-end rules, and allocation/alias lifetime. Use indices and object names.

4. **30 minutes:** trace three stream snippets: pending newline, formatting state, and recovery after a bad token. Explain where each output string goes.

5. **100-minute practice sitting:** take the accompanying 14-question mock without opening the answers. Suggested split: 15 minutes Part I, 35 Part II, 40 Part III, 10 checking. This timing is a practice suggestion.

6. **30 minutes:** mark it and redo only the missed or guessed items from scratch. For each mistake write the rule, the first incorrect state, and the corrected state.

### Before submitting any paper

- Check every option in a multiple-answer question independently; do not stop after finding one true statement.

- Read the actual exam's multiple-choice marking policy. This mock's exact-set rule is not an official rule.

- Check whether each question asks for output, final state, type, compilation status or an explanation.

- Re-check skipped &&/|| operands and integer division before conversion.

- Re-check *p++ versus (*p)++, one-past dereferences, and local-object lifetimes.

- Re-check trailing const on overrides and operators, plus return by value vs reference.

- Preserve spaces/newlines in output. Default booleans print as 0/1 unless boolalpha is used.

- Attempt all written subparts and leave a readable trace for partial credit.

### Source record and limitations

**Primary tutorial evidence:** Tutorials/C++ Concept Review/IMG_8041.HEIC through IMG_8071.HEIC (25 existing files; numerical gaps reflect the saved set). Exact mapping: 8041 slide 3; 8042-8049 slides 4-11; 8050-8053 slides 12-15; 8055 slide 16; 8056 slide 17; 8059/8063 slide 18; 8061/8064 slide 19; 8065 slide 20; 8067 slide 21; 8068 slide 22; 8069-8071 slides 23-25.

**Your TA notes:** C++ Concept Review.md is at the course root, not inside Exam Prep/Midterm. No additional concept-review file with that exact name was found there. The midterm folder contains CSC3002_Lectures_1-9_Revision_Guide.md/PDF and CSC3002_Midterm_Coverage_and_Focus.pdf; these supply the broader scope and practice-gap checklist.

**Supporting course material:** the nine original Lectures PDFs, Tutorial 4-Pointers, Arrays, Strings & Streams(1).pdf, and Tutorial 6-Class design.pdf. Refer to the existing revision guide's source map for lecture page references.

**Personal history:** chats titled "Create two interactive C quizzes", "make an interactive quiz to test my string knowledge based…", and "Review work and identify gaps". The first two provide your follow-up questions; the last contains grading of your submitted mock. No complete saved per-question interactive attempt log was available in the retrieved conversation history.

**Mock provenance:** original practice questions inspired by this evidence; not a reconstruction of unreleased exam questions. It concentrates on tutorial priorities and your gaps. A 14-question paper cannot exhaust all L1-9 material; retain the checklist on page 5.